Cubli Control

Build a reaction-wheel balancing cube and compare PD/LQR with Incremental Nonlinear Dynamic Inversion (INDI).

Start here

1. Solve and simulate a **one-wheel inverted pendulum** before designing the cube.
2. Size the motor and wheel together; CAD the frame, mounts, and wheel guard.
3. Build one restrained axis; verify IMU, motor torque control, and speed limits.
4. Establish PD/LQR balancing, then test INDI under the same disturbances.
5. Expand to three orthogonal wheels. Leave jump-up for a later stage.

Print brackets and fit-check parts, not an unqualified high-speed flywheel. Use a secured metal rotor, containment, and an emergency stop.

Build the documents

Pandoc and MiKTeX are used to turn the Markdown documents into LaTeX and PDFs. Run the VS Code task **Build Markdown PDFs** to create `build/latex/README.tex` and `build/latex/STUDY.tex`, plus `README.pdf` and `Ref/STUDY.pdf`. The **Watch Markdown PDFs** task builds both documents and rebuilds the affected PDF when either Markdown file is saved. Missing MiKTeX packages are installed automatically during compilation.

1D simulation

Start with one reaction wheel and a fixed, frictionless edge pivot. The parameters in Params.m are the CAD values from Table I of Bobrow et al., not measured prototype values.

Coordinates and the paper's missing offset

The simulation now uses the paper's Fig. 4 coordinates:

- θ_c = `thetab`: angle from world horizontal $+Y$ to a cube face.
- θ_w = `thetaw`: reaction-wheel angle relative to the cube.
- $\omega_w = \dot{\theta}_w$ = `thetawdot`: relative wheel speed.
- The center-of-mass line is at $\theta_c + \pi/4$.
- The unstable edge-balancing position is $\theta_c = \pi/4 = 45^\circ$, not zero.
- Positive motor torque accelerates the wheel and applies negative reaction torque to the body.

The paper gives

$$T = \frac{1}{2}\bar{I}_{cO}\dot{\theta}_c^2 + \frac{1}{2}I_{wG}(\dot{\theta}_c + \dot{\theta}_w)^2, \qquad V = m_c g d \sin\left(\theta_c + \frac{\pi}{4}\right).$$

Therefore,

$$\frac{\partial V}{\partial \theta_c} = m_c g d \cos\left(\theta_c + \frac{\pi}{4}\right).$$

The printed Eq. (7), Eq. (9), Eq. (11), and Fig. 5 show $m_c g d \cos\theta_c$ without the $\pi/4$. That expression conflicts with the paper's Eq. (6), Fig. 4 geometry, and its later statement that $\theta_c = 45^\circ$ is the unstable equilibrium. The simulation retains the paper coordinate and cosine gravity term, but restores the required offset:

$$\bar{I}_{cO}\ddot{\theta}_c + I_{wG}(\ddot{\theta}_c + \ddot{\theta}_w) = -m_c g d \cos\left(\theta_c + \frac{\pi}{4}\right),$$

$$I_{wG}(\ddot{\theta}_c + \ddot{\theta}_w) = -\tau_f(\dot{\theta}_w) + \tau.$$

Let $u = \tau - \tau_f(\dot{\theta}_w)$. The exact matrix model corresponding to the paper's Eq. (9) is

$$\begin{bmatrix} \bar{I}_{cO} & I_{wG} \\ 0 & I_{wG} \end{bmatrix} \begin{bmatrix} \ddot{\theta}_c \\ \ddot{\theta}_c + \ddot{\theta}_w \end{bmatrix} = \begin{bmatrix} -m_c g d \cos(\theta_c + \pi/4) \\ u \end{bmatrix}.$$

Solving it gives

$$\ddot{\theta}_c = \frac{-m_c g d \cos(\theta_c + \pi/4) - u}{\bar{I}_{cO}}, \qquad \ddot{\theta}_w = \frac{u}{I_{wG}} - \ddot{\theta}_c.$$

The second equation matters because θ_w is relative to the body. Paper Eq. (11) uses $\ddot{\theta}_w \gg \ddot{\theta}_c$ to approximate $\ddot{\theta}_w \approx u/I_{wG}$; the MATLAB script instead integrates the exact Eq. (9) dynamics. Motor friction uses paper Eq. (36) with the identified Table II values:

$$\tau_f(\dot{\theta}_w) = \mathrm{sign}(\dot{\theta}_w)\left(\tau_c + b_w|\dot{\theta}_w| + c_d\dot{\theta}_w^2\right),$$

$\tau_c = 2.46\times10^{-3}$ N m, $b_w = 1.06\times10^{-5}$ N m s/rad, $c_d = 1.70\times10^{-8}$ N m s^2/rad^2. `sign` is smoothed as $\tanh(\dot{\theta}_w/\omega_\epsilon)$, $\omega_\epsilon = 0.5$ rad/s, to avoid solver chattering at zero wheel speed. Optional pivot damping $-b_c\dot{\theta}_c$ is added to the body equation; the paper has none, so `bc = 0` by default.

Why the swinging cube barely loses energy

Motor friction is **internal**: it acts between wheel and body, and it only dissipates power $\tau_f \dot{\theta}_w$ when the wheel slips relative to the body. The paper has no pivot/edge friction, so the swinging body itself is conservative. With a small swing about the hanging position, the validated model loses about 15% of the swing energy in 10 s. Set `bc > 0` to model edge friction if your hardware decays faster.

In paper Eq. (11)/Fig. 5, the relative wheel acceleration is approximated as $(\tau - \tau_f)/I_{wG}$, independent of body motion. In that approximation, with $\tau = 0$ and $\dot{\theta}_w(0) = 0$, the wheel never slips, $\tau_f = 0$, and the body loses **no** energy at all.

Why constant torque barely moves the body with large `bw`

The body only feels the motor reaction while the wheel accelerates: $I_{wG}(\ddot{\theta}_c + \ddot{\theta}_w) = \tau - \tau_f$. Once friction equals the motor torque, $u = 0$ and the body sees no net torque. With a viscous `bw = 1e-2`, the wheel reaches τ/b_w (20 rad/s for 0.2 N m) in about $I_{wG}/b_w = 12.5$ ms, giving the body only a tiny impulse: θ_b moved about $\pm 1.3°$. With the paper's friction, the same torque spins the wheel to about 3000 rad/s over several seconds, and θ_b swings from $-115°$ to $-77°$. Also note gravity torque is $mgd \approx 0.88$ N m, so a constant hanging-position offset needs $\sin^{-1}(\tau/mgd)$; that is only 3.2° for 0.05 N m.

Validation results

Check	Result
Simulink vs `ode45`, 80° release, 0–2 s	$\max \lvert\Delta\theta_b\rvert = 6 \times 10^{-4}$ rad
Small-swing period about hanging	0.775 s vs theory $2\pi/\sqrt{mgd/\bar{I}_{cO}} = 0.771$ s
Hanging, 0.05 N m	body settles about 2.9° from hanging, near $\sin^{-1}(\tau/mgd) = 3.2°$
Upright 90°, zero torque	stays briefly, then falls (unstable equilibrium)

Set the initial conditions

Edit these values in Params.m:

```
thetab0 = deg2rad(90); % theta_c; upright is 90 deg when com_angle_offset = 0
thetabdot0 = 0;        % body speed, rad/s
thetaw0 = 0;           % wheel angle relative to body, rad
thetawdot0 = 0;        % wheel speed relative to body, rad/s
```

The Simulink integrators read these names directly; never type numbers such as `pi` into an integrator's initial condition. Then `x0 = [thetab0; thetabdot0; thetaw0; thetawdot0]`. Upright is `pi/2 - com_angle_offset` and hanging is `-pi/2 - com_angle_offset`. To observe falling, start a few degrees away from upright.

Simulink blocks

Run `Params` in the model's InitFcn callback (script must be on the MATLAB path). All angles entering trigonometric blocks are radians. Do not multiply an integrator output by `pi/180`; use `deg2rad` only for numeric initial values.

1. Subtract body speed from absolute wheel speed: $\dot{\phi} = s - \dot{\theta}$. In the four-state implementation this signal is directly `thetawdot`.
2. The `Friction` Fcn block computes paper Eq. (36) from `thetawdot`; subtract it from motor torque to obtain u.
3. Add `com_angle_offset` to `thetab` (`ComOffset` Bias block), apply `cos`, then multiply by `gravity_gain`. The body sum `Add` is $\tau_f - \tau - mgd\cos(\cdot) - b_c\dot{\theta}_c$; apply `body_accel_gain` to obtain $\ddot{\theta}_c$.
4. Integrate $\ddot{\theta}_c$ to $\dot{\theta}_c$, then to θ_c. Initial conditions: `thetabdot0`, then `thetab0`.
5. Apply `wheel_accel_gain` to u to obtain absolute wheel acceleration $\ddot{\theta}_c + \ddot{\theta}_w$. Subtract $\ddot{\theta}_c$ to obtain relative $\ddot{\theta}_w$, then integrate with `thetawdot0` and `thetaw0`.

The original pictured model used `cos(thetab*pi/180)`, making gravity nearly maximum at its zero initial state. With the listed values, `mgd` is about 0.8844 N m and $\bar{I}_{cO}$ is 0.0133125 kg m², so a 0.19 N m input at zero produces a very large acceleration. That explains the sharp scope drop; it is not an integration discontinuity.

MATLAB simulation and 3D animation

Run cubli_3d_compare.m. It performs these steps:

1. Runs `Params.m`.
2. Runs the Simulink model with `RelTol = 1e-8` and `MaxStep = dt`, then reads the **To Workspace** outputs `thetab`, `thetabddot`, `thetawdot`, and optionally `thetaw` (otherwise integrated from `thetawdot`).
3. Runs the exact nonlinear Eq. (9) model with `ode45` as a reference.
4. Resamples both on a uniform time grid and stores the Simulink result in the workspace table `measurements`.
5. Overlays Simulink and `ode45` for θ_b, $\ddot{\theta}_b$, relative $\dot{\theta}_w$, and total energy $\frac{1}{2}\bar{I}_{cO}\dot{\theta}_c^2 + \frac{1}{2}I_{wG}(\dot{\theta}_c + \dot{\theta}_w)^2 + mgd\sin(\theta_c + \delta)$.
6. Animates the selected source (`animationSource = "simulink"` or `"ode"`) about the fixed X edge. Red marks the pivot, RGB lines are body axes, magenta is the wheel axis, and the orange spoke shows wheel rotation.

Sending Simulink output to the animation

The model dynamic_of_reaction_wheel.slx already contains these **To Workspace** blocks (Save format `Timeseries`), each connected to a **radian** signal, not to a `180/pi` scope gain:

Variable name	Connect to
`thetab`	`Integrator1` (second body integrator), θ_b
`thetabdot`	`Integrator` (first body integrator), $\dot{\theta}_b$
`thetabddot`	`Gain` (`body_accel_gain`), $\ddot{\theta}_b$
`thetawdot`	`Integrator3` (first wheel integrator), relative $\dot{\theta}_w$
`thetaw`	`Integrator2` (second wheel integrator)

For `thetawdot` to be relative to the body, the wheel branch is:

1. `Add1 = tau - tau_f` (second input from the `Friction` block, **not** from the body sum).
2. `wheel_accel_gain` gives absolute acceleration $\ddot{\theta}_b + \ddot{\theta}_w = u/I_{wG}$.
3. `Add2` (`+-`) subtracts the `body_accel_gain` **output** $\ddot{\theta}_b$ (an acceleration, not the torque sum `Add`), giving relative $\ddot{\theta}_w$ for the wheel-speed integrator.

Integrator initial conditions are `thetabdot0`, `thetab0`, `thetawdot0`, and `thetaw0`. The gravity block uses `cos(thetab + com_angle_offset)`; the animation draws the center of mass at the same angle:

$$\mathbf{r}_G = \begin{bmatrix} 0 \\ d\cos(\theta_b + \delta) \\ d\sin(\theta_b + \delta) \end{bmatrix}, \qquad \delta = \texttt{com_angle_offset}.$$

With $\delta = 0$ (printed Eq. (7)), upright is $\theta_b = 90^\circ$; with $\delta = \pi/4$ (Eq. (6)), upright is 45°.

There is no floor collision, hard stop, motor saturation, or feedback controller in this open-loop model. Once the cube falls past the physical floor, the mathematical model continues rotating; add contact dynamics separately if impacts are required.

Study material

- ETH thesis summary, textbook reading, and math exercise
- Attached ETH thesis: Michael Muehlebach, 2018
- ETH Cubli project and papers
- Khalil, *Nonlinear Systems*, 3rd edition: author's companion site